Lab 4: Vibe Coding Pair Programming Log

Repository: SETAPESU26/39_endless-runner\nAssignment: Complete endless runner game features and fix broken collision bugs using LLM Vibe Coding.

Task 1: Refine Collision Detection

User Prompt:

The game speeds up forever with no limit, and once it is fast enough obstacles zip past the player without registering collisions. Investigate and fix the collision detection and speed ramp.

Vibe Coding Assistant Response:

Diagnosis:
1. Unlimited Speed Ramp: self.speed += 0.003 per frame without an upper ceiling (MAX_SPEED).
2. Tunneling / Missing Collisions: Collision detection was evaluated statically post-movement (colliderect). When speed > player width, an obstacle completely jumps over the player rect in a single frame.

Solution:
1. Introduce difficulty-based MAX_SPEED caps (e.g. 12.0 - 15.0 px/frame).
2. Implement Swept Bounding Box Continuous Collision Detection (CCD) in game_engine.py:
- Track prev_x for obstacles.
- Check if player vertical bounds overlap obstacle vertical bounds AND player horizontal bounds intersect the swept frame path [min(prev_x, x), max(prev_x+w, x+w)] covered by the obstacle in the frame.

Task 2: Implement Game Over Condition

User Prompt:

Add a proper screen that displays the final score once the player collides with an obstacle, then gracefully waits for input instead of just printing to the console.

Vibe Coding Assistant Response:

Solution:
1. Integrated state management (MENU, PLAYING, GAME_OVER).
2. Implemented
ender_game_over_screen() in GameEngine displaying:
- Modern styled translucent modal overlay & border.
- Final Score and High Score statistics.
- Interactive Replay and Difficulty Selection buttons on screen.
3. Updated main loop to wait gracefully for keyboard or mouse input without locking up or printing raw console output.

Task 3: Add Replay Option & Difficulty Selection

User Prompt:

After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard starting speed/spawn rate), or exit.

Vibe Coding Assistant Response:

Solution:
1. Created Difficulty Configurations:
- Easy: Start speed = 4.5, Spawn interval = 85, Max speed = 9.0
- Medium: Start speed = 6.0, Spawn interval = 70, Max speed = 12.0
- Hard: Start speed = 8.5, Spawn interval = 50, Max speed = 15.0

2. Added interactive menu & game-over UI buttons for mouse clicks and keyboard hotkeys (E for Easy, M for Medium, H for Hard, R to Replay, Q to Quit).
3. Added
eset_game(difficulty) for instant game restart with selected parameters.

Task 4: Add Sound Feedback

User Prompt:

Add basic sound effects for jumping, passing an obstacle (scoring), and the game-over moment.

Vibe Coding Assistant Response:

Solution:
1. Programmatically synthesized 8-bit sound effects using standard library PCM generation (wave & struct):
   - ssets/sounds/jump.wav: Pitch sweep (250Hz -> 650Hz)
   - ssets/sounds/score.wav: Dual-tone arpeggio (523Hz -> 659Hz)
   - ssets/sounds/game_over.wav: Descending crash tone (400Hz -> 100Hz)
2. Integrated pygame.mixer.Sound in GameEngine with safe exception handling so sounds trigger on events (Jump, Score, Game Over) and fall back gracefully if no audio output device is attached.